

SECAUDIT - Security Audit Standard (Public Edition)

SECURITY AUDIT / PUBLIC REMEDIATION RECORD

Status: Living document. This is the standard every SnapSmack security audit works from. **Maintained by:** the SnapSmack maintainer, with two or more independent AI reviewers. **Why it exists:** Early in the project, a security pass shipped as a *diff review* - it checked the lines that had changed, not the doors the code left open. That gap is real and it bites. This standard exists so that failure is never repeated by accident.

Read this before starting any audit. If a pass does not meet the bar below, it is not an audit - it is a spot-check, and it must be labelled as one.

This is the public edition of our internal audit standard. It is method only: it describes how we audit, not the current state of any specific control. Security audits are defensive work on our own software; this document contains no exploit code and never will.

0. The one-line rule

Audit the doors the code opened, not just the lines the code changed. A diff review answers "is this change safe"; an audit answers "is this attack surface defended." They are not the same, and the difference bites.

1. When to run a full audit vs a spot-check

- **Full audit** (this standard applies): before a public milestone; when a new boundary or data flow is introduced (a new tool, a new sync channel, a new file format, a new external process); when a component gains write/exec/credential surface; when the maintainer commissions one.
- **Spot-check** (lighter, must be labelled as such): a single reversible change on a branch, reviewed for regressions. Never call a spot-check an "audit," and never file a spot-check as a close of a boundary it did not examine.

2. Numbering & filing

- **One monotonic integer** per audit, never reused. Next number = (highest existing) + 1.
- Check the folder before you pick a number** - picking one from memory is how collisions happen.
- **Filename:** YYYY-MM-DD-NNN-short-slug.md.
 - **The working source is kept private.** The **published artifact is a signed document**, released through a signed channel - a **human-gated** step, taken only after the relevant fixes are in a released build. **An agent never publishes.**
 - The process document sorts to the top of the audit folder so it is unmistakable.

3. The method - non-negotiable rules

1. **Chokepoint coverage, not diff coverage.** Enumerate every boundary where untrusted data crosses (section 4) and examine each. Coverage must be deliberate, never accidental.
2. **Two or more INDEPENDENT adversarial reviewers, never a single self-audit.** Reviewers

work separately, then cross-check; a severity disagreement is itself a finding, resolved with the maintainer present. In our own use, moving from one reviewer to several has surfaced findings a single narrow pass missed. The guardrail earns its keep.

3. **Rate by FAIL-CLOSED enforcement at the boundary - not "today's caller is safe."** Harden

where untrusted input enters, not where today's one caller happens to be OK. A control that is only safe because of who calls it today is not a control.

4. **Enforce at the ACTING boundary, not the asking one.** A client-side check ("the tool

won't do it") is advisory - a modified client, a future bug, or another tool bypasses it. The close lives where the action happens: the server rejects the out-of-scope write; the desktop verifies the signature before acting on config. Never file an asking-side control as an enforcing close. (An early pass in our own history mistook a client-side check for a close; the enforcing boundary was elsewhere. That lesson is baked into this rule.)

5. **Evidence for every conclusion - safe or unsafe.** Cite file:line. A closed item needs

proof as much as an open one. "Looks fine" is not a finding.

6. **Never claim "done" on code presence or a headless build.** Controls that carry code

execution (content-security policy, updaters, decoder gates) require a build + live test as part of the audit, not after it. If you can't run the live test, the item stays OPEN with the exact test named - you do not ship a code-execution control blind.

7. **Ground in live code before writing a finding.** Live code supersedes any spec or memory.

Name the files you read.

8. **No exploit code.** Findings describe the vulnerability CLASS, the failure mode in one

concrete sentence, and the fail-closed control. This is defensive work on our own software.

4. The chokepoint map (the coverage checklist)

Every full audit confirms each of these is fail-closed, or files it open. This map is a coverage checklist - a list of the doors we check on this class of software, not a statement about which are locked.

1. **Untrusted image ingress** - image decoders operating on downloaded, project-referenced,

or federation-supplied bytes; external decoders - instruction hygiene AND parser containment.

2. **Config / manifest / profile ingestion** - does a tool ACT on server- or file-supplied

URLs/paths without validation or a signature? (Config-as-action.)

3. **JavaScript / IPC into desktop webviews** - content-security policy, capability/command

scoping, HTML-injection sinks.

4. **Update & restore channel** - is a downloaded or restored artifact signature-verified

BEFORE it is trusted or executed? Archive path-traversal, library planting.

5. **Credential & token store at rest** - real encryption (vault/keychain) vs recoverable

encoding.

6. **Desktop -> server upload** - the server parser is the target; server-side request forgery

via client-supplied URLs.

- **Cross-cutting:** AI model output is DATA, never an instruction a tool acts on; prompts are

untrusted wherever they can be influenced; a sync channel must not propagate unvalidated content that a tool then acts on.

- **Exotic** (included because responsible hardening anticipates the unlikely): XML external

entities, polyglot/extension confusion, auto-processing of an untrusted media folder, shared-state contamination, and deserialization/eval on untrusted data.

5. Severity - rank by reach

Rank findings by **how close the vector gets to code execution or credential/full-site compromise**, not by abstract score. Order the report that way. A "LOW" that is a false-green - a status that reports success when the thing did not happen - is still worth flagging: it is the exact class our "done = verified" standard exists to kill.

6. Sign-off tiers

- **Single-reviewer close.** A reversible, in-scope, boundary-hardening fix with a regression

test, verified - the maintainer or a single AI reviewer may close it.

- **Full sign-off (maintainer present).** Anything DB-mutating or format-changing; the

authentication boundary; the credential-at-rest model; config signing; anything touching where a person's data comes to rest. No single reviewer closes these - the maintainer and the independent reviewers sign off together, with the maintainer present.

- **The MEMENTO MORI floor.** MEMENTO MORI preserves the archives of photographers who have

died - the most irreplaceable trust the project holds, kept for people who can no longer consent to a risk. Any tool that touches those archives clears the project's full security bar *before* it touches them, not after. The people carrying that risk can't say no, so the floor is higher.

7. Output format (the house SECAUDIT template)

Every audit document carries: a metadata table (Audit ID, Date, Severity with rationale, Scope, Method, Status, Reporter, Related, Disclosure); **1. Summary**; **2. Method** (what was examined and how - reviewers, files, grounding); **3. Findings** (ranked by reach, each with severity, `file:line`, a one-sentence failure mode, the fail-closed fix, and status); **4. Verified safe** (with evidence - closed items carry proof); **5. Recommendations / open items** (each with its sign-off tier); **6. Closure checklist.** When a pass supersedes or corrects an earlier one, say so, and document the shortcoming of the earlier pass.

8. Pre-flight checklist (run before filing)

- [] Number checked against the folder; filename YYYY-MM-DD-NNN-slug.md.
- [] Every one of the 6 chokepoints examined, or explicitly scoped out with a reason.
- [] Two or more independent reviewers; cross-checked; disagreements resolved.
- [] Every finding (safe or unsafe) cites `file:line`.
- [] Each finding rated by reach-to-code-execution; report ordered that way.
- [] No asking-side control filed as an enforcing close.
- [] Code-execution controls that need a build + live test are OPEN with the test named - not

claimed done.

- [] Sign-off tier assigned to each open item.
- [] Any superseded or corrected prior pass documented.
- [] Publication left as a maintainer-gated release step (a human, never an agent); the

working source stays private.